

Graphes Multiplexes

Projet de fin d'études

Thomas Castaigna ; Quentin Bret ; Quentin Chaumette ; Christophe Say-Liang-Fat ; Odrian
Tarrieu ; Corentin Drezen

1 TABLE DES MATIÈRES

1	Table des matières	i
2	Présentation du projet	4
3	Description des besoins	4
3.1	Besoins fonctionnels.....	4
3.2	Besoins non fonctionnels.....	5
3.3	Scénario utilisateur typique	6
4	Outils utilisés	6
5	Fonctionnement	7
6	Architecture	12
6.1	Architecture de la partie Backend (Django)	13
6.1.1	Applications Django	13
6.1.2	Autres dossiers	14
6.2	Architecture de la partie Frontend (React).....	14
6.3	Base de données.....	15
6.4	Flux global	15
7	Tests	15
8	Tactiques d'utilisation d'outils d'IA générative	16
8.1	Génération de snippets de code.....	16
8.2	Assistance à la refactorisation	16
8.3	Génération et validation de tests	16
8.4	Recherche d'erreurs et débogage	17
9	Optimisation des performances.....	17
9.1	Méthodologie	17
9.2	Déroulement	18
9.3	Points délicats	23
10	Analyse des sprints	24
10.1	Sprint 1 (17 au 28 février 2025)	24
10.2	Sprint 2 (10 au 16 mars 2025).....	26
10.3	Sprint 3 (17 au 21 mars 2025).....	27
10.4	Sprint 4 (24 au 26 mars 2025)	28
11	Retours sur Scrum et son application	29
12	Bibliographie	30
13	Annexe	31

13.1	Tableau des tests réalisés côté backend	31
13.2	Tableau des tests réalisés côté frontend	33
13.3	Diagramme de flux de l'application	34

Graphes Multiplexes

Projet de fin d'études

Thomas Castagna ; Quentin Bret ; Quentin Chaumette ; Christophe Say-Liang-Fat ;
Odrian Tarrieu ; Corentin Drezen

2 PRÉSENTATION DU PROJET

Un graphe multiplexe est un graphe dont les sommets sont répartis entre plusieurs couches. Les arêtes peuvent donc soit être contenues intégralement dans une couche, soit relier deux nœuds appartenant à des couches différentes.

Les réseaux sur lesquels nous travaillons utilisent des nœuds appartenant à différentes couches pour représenter des entités biologiques telles que des gènes, des protéines et des maladies et les arêtes pour représenter des relations de coexpression ou de corrélation. L'objectif de ce projet est de construire un outil capable d'appliquer un algorithme de marche aléatoire à partir d'un ou plusieurs nœuds appelés "graines" à l'intérieur de ces graphes (nous avons travaillé à partir de l'algorithme itRWR¹ basé sur MultiXRank²) et d'afficher le sous-graphe résultant de cette marche aléatoire, c'est-à-dire en ne représentant que les nœuds ayant reçu la note de marche aléatoire la plus élevée.

3 DESCRIPTION DES BESOINS

3.1 BESOINS FONCTIONNELS

- I. Sélection et Import de Fichiers (US₁)
 - a. Permettre l'importation de fichiers locaux via un assistant ou via une fenêtre d'ouverture de fichier.
 - b. Valider le format des fichiers importés (TSV contenus dans un ZIP de structure appropriée).
- II. Choix de Couches dans un Graphe (US₂)
 - a. Afficher les couches disponibles après sélection du fichier.
 - b. Permettre la sélection d'une couche.
- III. Sélection d'un Nœud de Départ (US₃)
 - a. Afficher la liste des nœuds disponibles dans la couche choisie.

¹ (Baptista, Beust, & Baudot, itRWR: MultiXrank-based Community Identification with Iterative Random Walk with Restart, s.d.)

² (Baptista, González, & Baudot, Universal multilayer network exploration by random walk with restart, 2022)

- b. Permettre la recherche par nom.
 - c. Permettre l'entrée textuelle.
 - d. Permettre de sélectionner une liste de nœuds comme graines pour le départ de la marche aléatoire.
- IV. Configuration de l'Algorithme (US₄)
 - a. Permettre la saisie et la validation des paramètres de l'algorithme de marche aléatoire.
 - i. Nombre d'itérations (profondeur du parcours)
 - ii. Restart (la probabilité réinjectée dans les graines à chaque nouvelle itération)
 - iii. Nombre de nœuds à afficher pour chaque couche dans le graphe de sortie
 - b. Proposer des paramètres par défaut appropriés (sur conseil des MOA).
- V. Exécution et Visualisation 2D (US₅)
 - a. Lancer l'algorithme itRWR avec les paramètres fournis.
 - b. Afficher les résultats sous forme de graphe 2D interactif.
- VI. Consultation d'Informations sur un Nœud (US₆)
 - a. Permettre de survoler un nœud pour afficher ses informations détaillées dans une infobulle (nom, couche, score, voisins).
- VII. Sauvegarde et export des Résultats (US₇)
 - a. Sauvegarder les résultats de la marche aléatoire dans une base de données.
 - b. Permettre le téléchargement des résultats au format ZIP.
 - c. Permettre la suppression d'un jeu de résultats de la base de données.
 - d. Permettre la consultation ultérieure du graphe de sortie.

3.2 BESOINS NON FONCTIONNELS

- VIII. Performance :
 - a. L'application doit pouvoir gérer des graphes de la taille couramment manipulée en bio-informatique (jusqu'à 5 couches, chacune comptant entre 100 et 100 000 nœuds, et plusieurs centaines de milliers d'arêtes)
 - b. L'algorithme utilisé doit être optimisé³ de manière à proposer un temps raisonnable d'exécution pour des graphes de cette taille (temps de l'ordre des quelques minutes pour 10 itérations).
- IX. UX/UI :
 - a. Interface épurée et intuitive avec des infobulles explicatives.
 - b. L'interface utilisateur doit être intégralement en anglais.
- X. Fiabilité

³ Voir la partie Optimisation des performances

- a. Les modifications apportées à l'algorithme de marche aléatoire ne doivent pas causer de différences par rapport aux résultats obtenus avec itRWR.
- b. Nos améliorations de l'algorithme doivent être documentées afin de pouvoir être étudiées par des équipes de bio-info.

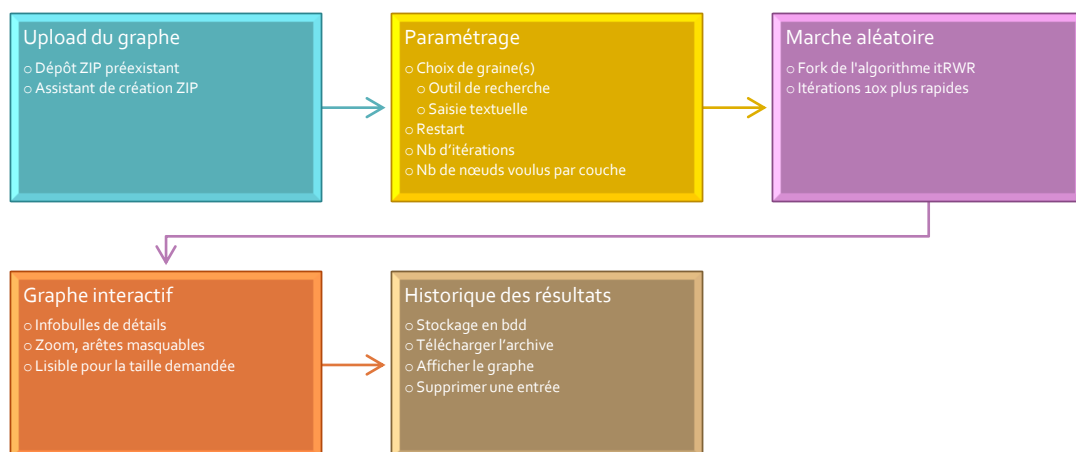
XI. Lisibilité

- a. Les graphes de sortie doivent être lisibles avec un nombre de couches pouvant aller jusqu'à 5 et comptant jusqu'à 10 nœuds chacune (i.e. les superpositions sont à éviter)
 - i. Les nœuds doivent avoir une taille variable selon le score attribué par la marche aléatoire
 - ii. Les arêtes doivent, au possible, ne pas se superposer
 - iii. Les arêtes doivent avoir des couleurs différentes suivant leurs couches d'origine et d'arrivée

XII. Compatibilité

- a. L'application doit pouvoir être exécutée sur des postes de travail Mac et Linux.

3.3 SCÉNARIO UTILISATEUR TYPIQUE



4 OUTILS UTILISÉS

Étant donné que les algorithmes de marche aléatoire utilisés (ITRWR et MultiXRank) étaient écrits en Python, nous avons décidé d'écrire notre backend dans ce même langage, en faisant appel au framework MVC Django.

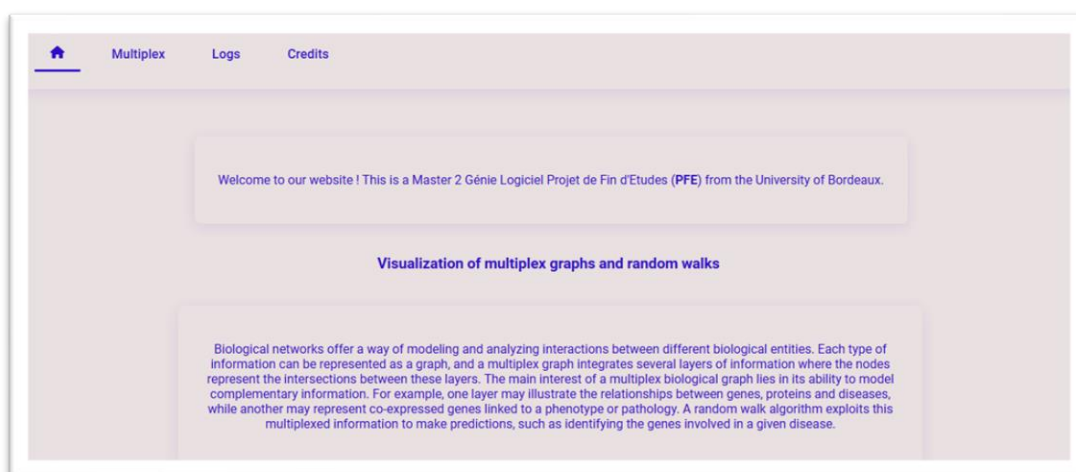
Pour le frontend, nous avons utilisé le framework React. Nous l'avons déjà utilisé dans nos projets précédents et il convenait à nos attentes et espérances.

Pour l’affichage du graphe de sortie, nous avons fait appel à la bibliothèque Plotly. Celle-ci avait une grande flexibilité nous permettant d’implémenter toutes les fonctionnalités supplémentaires demandées concernant l’affichage des graphes.

Le programme a été développé comme une application web, disposant d’un backend et d’un frontend, comme demandé par les MOA. Toutefois, elle n’est pas *a priori* amenée à être hébergée sur des serveurs web mais exécutée via docker sur des postes de travail Mac et Linux. En réalité, une application native aurait aussi bien convenu.

5 FONCTIONNEMENT

Quand on lance l’application, on arrive sur la page d’accueil (onglet ). Elle est présente à titre informatif : elle contient le sujet proposé par les chercheuses, avec notamment une explication de ce que l’application est censée faire. L’interface utilisateur est entièrement en anglais, comme requis par les MOA.



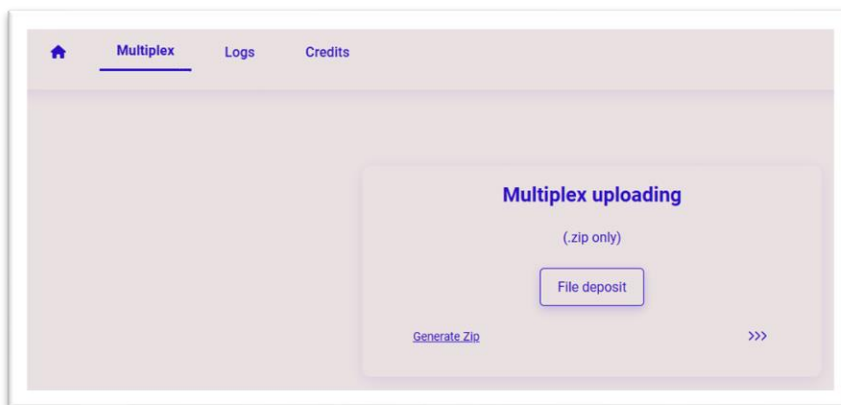
Dans la barre de navigation en haut de la fenêtre, plusieurs onglets sont accessibles : , **Multiplex**, **Logs** et **Credits**.

Nous nous concentrerons tout d’abord sur l’onglet **Multiplex**. C’est dans cet onglet qu’il est possible de charger le graphe sur lequel l’utilisateur souhaite travailler.

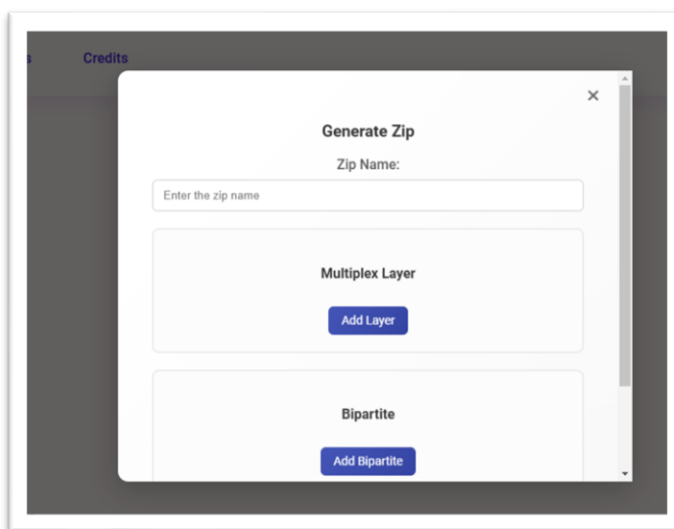
Les graphes attendus en entrée doivent être sous la forme d’une archive ZIP contenant

- Un dossier `multiplex` contenant
 - Des dossiers numérotés (à partir de 1) correspondant aux couches, chacun contenant les fichiers TSV décrivant les relations entre nœuds d’une même couche

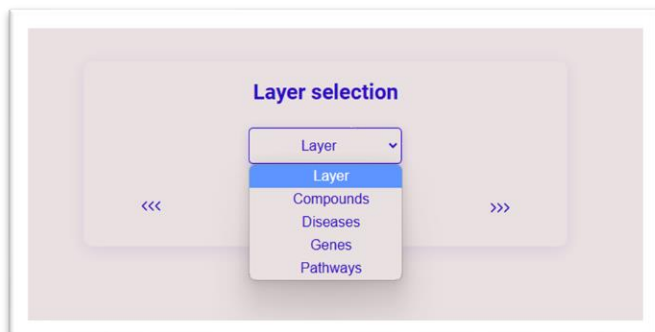
- Un dossier **bipartite** contenant des fichiers **M_N.tsv**, où M et N sont des entiers, décrivant les relations entre les nœuds de la couche M et ceux de la couche N respectivement
- Un fichier **layer_name.tsv**, contenant deux colonnes, qui désigne pour un numéro donné le nom de la couche associée.



Si l'utilisateur n'a pas déjà préparé une archive, nous pouvons l'y guider pas à pas grâce à un assistant (wizard).



Après mise en ligne de l'archive, l'utilisateur est amené à choisir dans quelle couche il souhaite sélectionner une ou plusieurs graines pour l'exécution de l'algorithme.



La liste des nœuds disponibles dans cette couche est ensuite chargée (cela peut prendre quelques secondes car la liste des nœuds n'est stockée nulle part dans la structure de données du graphe. Seules les arêtes sont stockées et elles peuvent être au nombre de plusieurs centaines de milliers).

L'utilisateur a ensuite à disposition un formulaire permettant le choix de celles-ci ainsi que le paramétrage de l'algorithme.

Ici, par exemple, l'utilisateur a sélectionné la couche **diseases**, puis les graines **acquired immunodeficiency syndrome** (SIDA) et **attention deficit hyperactivity disorder** (TDAH). La marche aléatoire sera exécutée avec une seule itération et une valeur restart de 0,7⁴. 10 nœuds seront affichés pour chaque couche du graphe de sortie. Une option de filtrage par nom est aussi proposée. Il est toute fois possible de rentrer manuellement les noms des graines dans le champ **Seeds** du formulaire. Chaque graine doit être séparée par une virgule. La sélection de graine nécessite une validation d'un popup de sélection.

⁴ Voir le besoin IV.a et (Baptista, González, & Baudot, Universal multilayer network exploration by random walk with restart, 2022) pour une explication de la signification de ces paramètres

Le popup s'affiche en rouge si la sélection n'est pas valide (au moins une graine introuvable dans la couche sélectionnée) et la validation du popup de sélection supprime automatiquement de celle-ci l'ensemble des graines invalides. La suppression de l'ensemble des graines sélectionnées est possible en cliquant sur la croix du popup de sélection.

En cliquant sur **Start**, on lance l'exécution.



Une fois l'exécution de la marche aléatoire terminée, un nouvel onglet s'ouvre affichant le graphe interactif des résultats de la marche aléatoire (si les pop-ups sont autorisés par les paramètres du navigateur de l'utilisateur).

Dans nos graphes, les couches sont affichées comme des tranches. Les graines sont représentées par des carrés, les autres nœuds par des ronds. Les nœuds ont une taille proportionnelle à leur score attribué par l'algorithme de marche aléatoire. Les arêtes sont grises si internes à une couche, reçoivent un code couleur sinon ; chaque catégorie d'arêtes peut être masquée ou affichée séparément. Il est également possible de zoomer sur toute partie du graphe. En survolant un nœud, une infobulle est affichée contenant notamment son score ainsi que la liste de ses voisins. Afin d'éviter les superpositions

d'arêtes, une légère rotation est appliquée à chaque couche pour minimiser les alignements de nœuds.

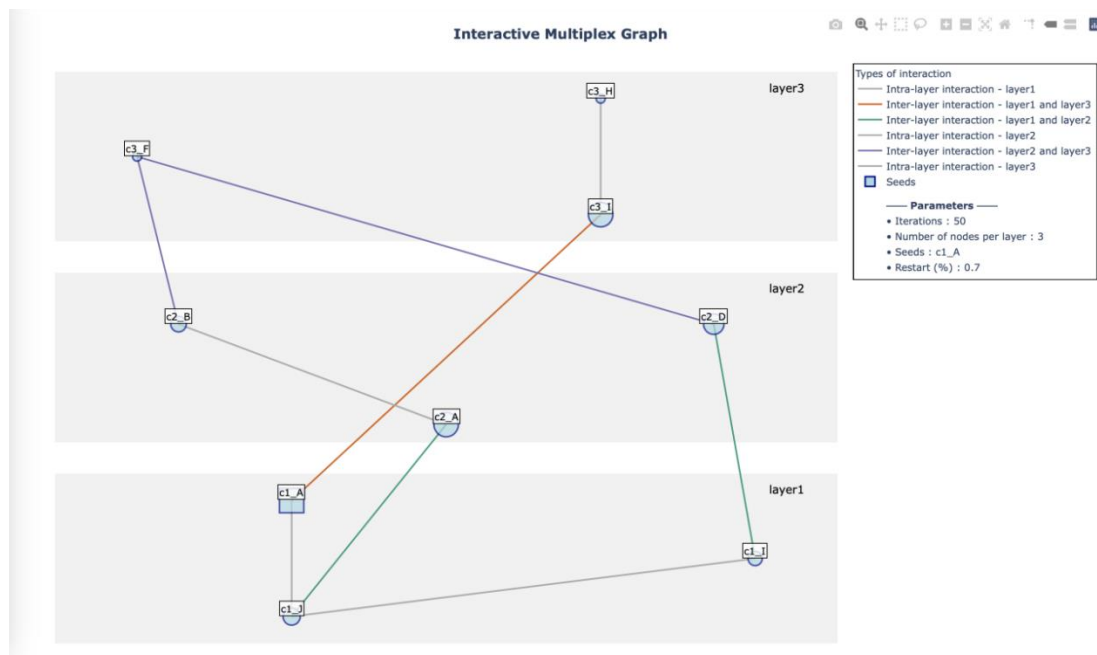


Figure 1: Exemple trivial de graphe de sortie

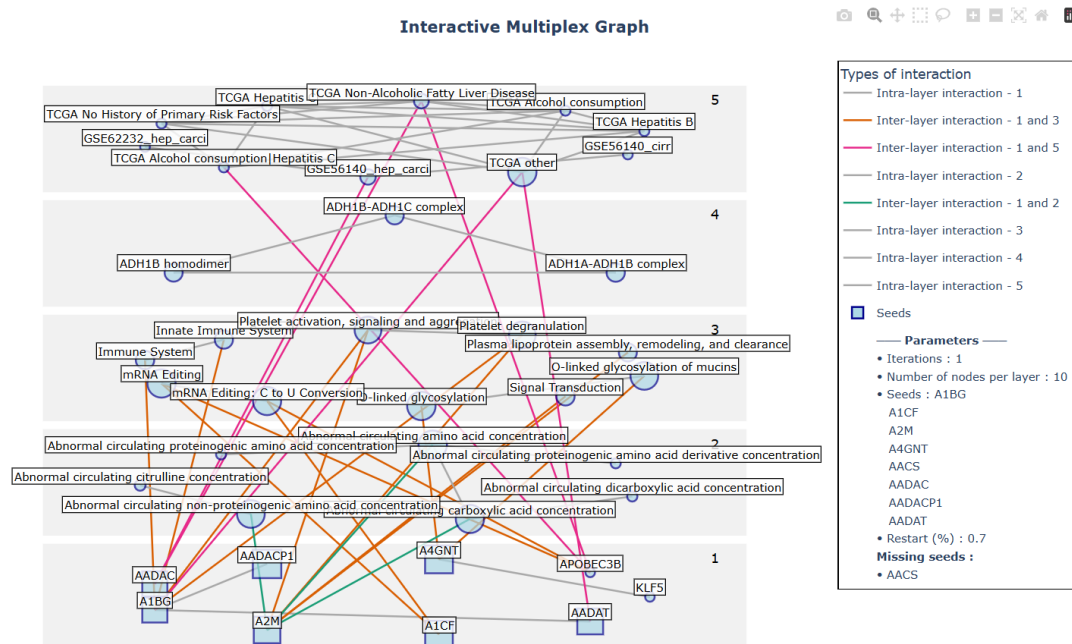
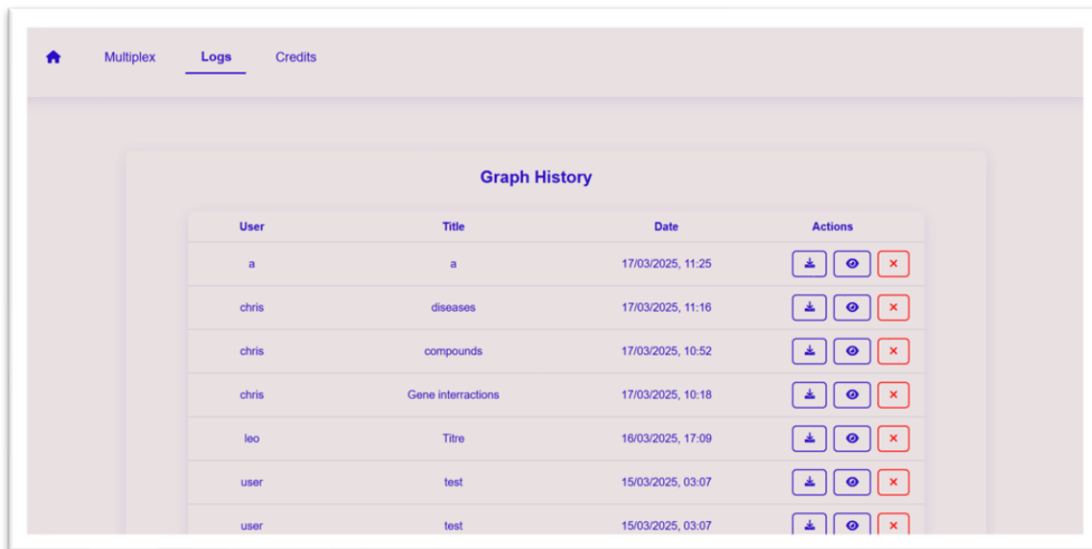


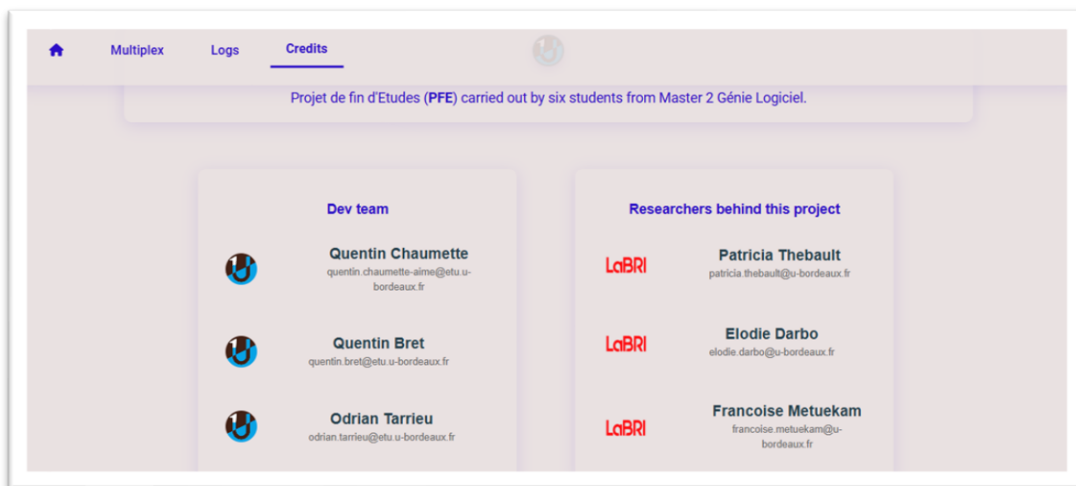
Figure 2: Exemple non-trivial de graphe de sortie

Nous proposons dans l'onglet **Logs** une archive des résultats de chaque marche aléatoire effectuée jusqu'ici, qui peuvent être affichés, téléchargés (l'archive entière est téléchargée, contenant les données sources, les résultats de la marche aléatoire et le graphe HTML) ou supprimés.



User	Title	Date	Actions
a	a	17/03/2025, 11:25	[Download] [View] [Delete]
chris	diseases	17/03/2025, 11:16	[Download] [View] [Delete]
chris	compounds	17/03/2025, 10:52	[Download] [View] [Delete]
chris	Gene interactions	17/03/2025, 10:18	[Download] [View] [Delete]
leo	Titre	16/03/2025, 17:09	[Download] [View] [Delete]
user	test	15/03/2025, 03:07	[Download] [View] [Delete]
user	test	15/03/2025, 03:07	[Download] [View] [Delete]

L'application dispose également d'un onglet **Credits**, créditant les développeurs comme les chercheuses à l'initiative de ce projet.



Projet de fin d'Etudes (PFE) carried out by six students from Master 2 Génie Logiciel.

Dev team



Quentin Chaumette
quentin.chaumette-aimé@etu.u-bordeaux.fr



Quentin Bret
quentin.bret@etu.u-bordeaux.fr



Odrian Tarrieu
odrian.tarrieu@etu.u-bordeaux.fr

Researchers behind this project



Patricia Thebault
patricia.thebault@u-bordeaux.fr



Elodie Darbo
elodie.darbo@u-bordeaux.fr



Francoise Metuekam
francoise.metuekam@u-bordeaux.fr

6 ARCHITECTURE

L'application se décompose en trois grandes couches :

- **Base de données** : On utilise SQLite pour stocker les informations sur les historiques de graphes (utilisateur, titre, paramètres, graphe en format JSON).
- **Backend** : développé avec le framework Django (Python), il gère la logique de traitement (import de l'archive ZIP, exécution des algorithmes de marche aléatoire, enregistrement des logs et génération du graphe résultant).
- **Frontend** : développé en React, il propose l'interface utilisateur permettant de configurer et lancer les marches aléatoires, d'afficher les sous-graphes obtenus et de consulter les historiques de résultats.

Le déploiement est orchestré via un fichier `docker-compose.yml` qui décrit la mise en service des conteneurs (backend, frontend, base de données).

6.1 ARCHITECTURE DE LA PARTIE BACKEND (DJANGO)

La racine du dossier backend contient :

- `manage.py` : point d'entrée standard pour les commandes Django (exécuter le serveur de développement, gérer la base de données, etc.).
- `Dockerfile`, `.dockerignore`, `.gitignore`, `setup-venv.sh` : fichiers relatifs à la conteneurisation et à la configuration de l'environnement Python.
- `requirements.txt` : liste des dépendances Python (Django, bibliothèques pour la manipulation de graphes, etc.).

6.1.1 Applications Django

Le backend est organisé en plusieurs « applications » Django, chacune spécialisée dans un volet fonctionnel :

6.1.1.1 App

- `graph/` : contient le code relatif à la représentation et à la manipulation des graphes multiplex (lecture, formatage, création d'objet `MultiplexGraph`, etc.).
- `itRWR/` : implémente l'algorithme de marche aléatoire *Iterative Random Walk with Restart* (ITRWR) en se basant sur l'algorithme itRWR déjà existant (ajout de personnalisations).
- `multixrank/` : contient l'implémentation de MultiXRank en version optimisée (réduction du temps d'exécution).
- `utils/` : regroupe les fonctions d'utilité générale comme la création du ZIP, la génération des fichiers de configuration nécessaire à l'exécution de l'algorithme itRWR, la récupération information sur le graphe (nom des couches, nœuds présent dans une couche ...) etc.
- `visualization/` : gère la génération des représentations graphiques interactives (Plotly) servant au frontend pour afficher le sous-graphe résultant.

6.1.1.1.1 Fichiers standards Django

- `views.py` : contient des fonctions et classes `views` qui répondent aux requêtes HTTP (sauvegarder le ZIP, exécuter l'algorithme etc.).
- `urls.py` : déclare les points d'entrée (URL) de notre application.

6.1.1.2 History

Gère l'historique des calculs effectués (logs, configurations, résultats).

- `models.py` définit la table en base concernant l'historique.
- `serializers.py` employé afin de pouvoir exposer au format JSON via des API REST.
- `templates/` : Page HTML où sera stocké le sous-graphe résultant

6.1.2 Autres dossiers

- `samples/` : stocke le ZIP qui est utilisé pour le traitement courant, ainsi que les résultats temporaires et archives de résultats.
- `tests/` : organise les tests unitaires et les données de test associées.

La communication entre ces différentes applications Django s'effectue via les appels de vues et l'utilisation commune de la base de données SQLite.

6.2 ARCHITECTURE DE LA PARTIE FRONTEND (REACT)

Le dossier frontend contient l'application React. On y retrouve :

- `Dockerfile`, `.env.docker` : fichiers de configuration pour la conteneurisation et les variables d'environnement spécifiques au frontend.
- `package.json`, `package-lock.json` : configuration Npm, déclarant les dépendances (React, Plotly.js, etc.).
- `src/` : dossier principal du code source React. On y trouve :
 - `index.js` et `App.js` : points d'entrée de l'application React.
 - `components/` : ensemble de composants réutilisables, par exemple :
 - `Graph.jsx` et `GraphFromLog.jsx` : affichage du sous-graphe interactif pour la partie History (via Plotly).
 - `Logs.js`, `History.jsx` : composants pour lister et consulter l'historique des résultats.
 - `MultiplexLOAD.js`, `MultiplexCHOICE.js` et `MultiplexMA.js` : gestion des formulaires et interactions spécifiques à la configuration des marches aléatoires et à l'affichage des résultats en sortie après exécution (sans passer par l'historique).
- `apiService.js` : centralise les appels HTTP au backend (envoi des paramètres, récupération des résultats, etc.).
- `public/` : ressources statiques

Le frontend communique avec le backend via des requêtes HTTP. Les points d'API côté Django renvoient des réponses (JSON) que les composants React traitent.

6.3 BASE DE DONNÉES

Le projet utilise par défaut SQLite, embarqué dans Django. L'application `history` stocke l'ensemble des informations d'historique (paramètres choisis, date d'exécution, sous-graphe résultat en format JSON, etc.).

Cette organisation permet de garder en mémoire l'ensemble des sous-graphes obtenus afin de faire éventuellement des comparaisons.

6.4 FLUX GLOBAL

Importation d'une archive ZIP via le frontend : le ZIP est envoyé au backend, validé, et stocké dans le dossier `samples/`.

Configuration de la marche aléatoire : l'utilisateur, via l'interface React, sélectionne la couche de départ, les graines et les paramètres (probabilité de restart, nombre d'itérations, etc.).

Exécution de l'algorithme (ITRWR) côté backend :

- Création du graphe multiplex dans `app/graph` à l'aide des informations présentes dans le ZIP importé.
- Exécution de l'algorithme dans `itrwr/`.
- Génération du sous-graphe résultant dans `visualization/`
- Visualisation : le backend envoie la représentation (dictionnaire utilisable par Plotly) au frontend, où `MultiplexMA.js` se charge de l'affichage interactif.
- Enregistrement des logs : l'application `history` conserve la trace (paramètres utilisés, date et heure, identifiants, sous-graphe en format JSON etc.). L'utilisateur peut ensuite les consulter (cette fois, grâce à `Graph.jsx`) ou télécharger l'archive.

7 TESTS

Pour l'écriture des tests, nous avons utilisé le framework `unittest`, qui est directement intégré à Django. Ceci nous permet d'exécuter l'ensemble des tests définis dans les fichiers `test_[fonction].py` du dossier `backend/app/tests`.

Nous avons implémenté les tests unitaires des fonctions après les fonctions elles-mêmes, et nous avons utilisé la bibliothèque `unittest.mock`, pour éviter de faire appel aux dépendances externes dans nos tests d'intégration.

Nous avons rencontré des difficultés pour écrire les tests au début, car jusqu'aux deux dernières semaines du projet, les fonctions à tester évoluaient en permanence. Ce n'est qu'après avoir établi l'API connectant le frontend au backend que l'architecture du projet s'est stabilisée. En rétrospective, il aurait fallu se fixer le comportement attendu de chaque fonction plus tôt, pour éviter les retards dans les tests.

Nos tests concernent les fonctionnalités de l'application web telles que la visualisation des graphes, la création de l'historique des résultats, la génération des fichiers TSV contenant les résultats de la marche aléatoire (scores attribués à chaque nœud), l'analyse des archives ZIP importés en entrée, et de celles générées en sortie.

Une liste complète des tests réalisés, avec leurs domaines d'application, d'utilisation, et leurs descriptions, est disponible en 13.

8 TACTIQUES D'UTILISATION D'OUTILS D'IA GÉNÉRATIVE

L'intégration d'outils d'IA générative dans notre workflow a principalement visé à accélérer certaines étapes de développement et à consolider la fiabilité du code. Nous avons exploité ces outils pour faciliter la production et la validation du code. Voici comment nous les avons mobilisés :

8.1 GÉNÉRATION DE SNIPPETS DE CODE

Création rapide de prototypes : Nous avons pu demander à l'IA de produire des exemples de code lorsque nous explorions une nouvelle fonctionnalité ou un nouveau design pattern. Cela nous a permis de gagner du temps, en ayant immédiatement un squelette de solution à partir duquel itérer.

Idées d'implémentation : Dans certains cas, l'IA proposait plusieurs variantes de code pour résoudre un problème donné. Cela nous offrait une large palette de choix et nous aidait à comparer rapidement différentes approches avant de converger vers la plus appropriée pour notre application.

8.2 ASSISTANCE À LA REFACTORISATION

Amélioration de la lisibilité : Il nous est arrivé de solliciter l'IA pour suggérer des noms de variables ou de fonctions plus cohérents, et pour organiser le code de manière plus structurée (réduction de code dupliqué, extraction de méthodes communes, etc.).

8.3 GÉNÉRATION ET VALIDATION DE TESTS

Tests unitaires : Lorsqu'une nouvelle fonctionnalité était développée, nous avons parfois utilisé l'IA pour obtenir des propositions de tests (e.g. structure des assertions,

couverture des cas limites). Nous gagnions alors un aperçu de tests possibles pour mieux garantir la robustesse du code.

8.4 RECHERCHE D'ERREURS ET DÉBOGAGE

Analyse d'erreurs : En cas de message d'erreur obscur, nous décrivions la situation à l'IA en lui fournissant l'exception ou le stack trace, pour rapide diagnostic ou pistes de résolution.

L'utilisation d'outils d'IA générative dans notre projet s'est donc concentrée sur la production de code, la refactorisation, la génération de tests et la résolution rapide de problèmes techniques. Bien qu'elle ne se soit pas substituée à l'expertise et à la relecture humaine, cette approche nous a permis de gagner du temps et d'optimiser notre workflow de développement, tout en renforçant la qualité du code grâce à une vue complémentaire sur nos pratiques de programmation.

9 OPTIMISATION DES PERFORMANCES

Un des besoins non fonctionnels (VIII.b) de notre projet était d'avoir une exécution de l'algorithme de marche aléatoire suffisamment performante. En effet, avec les graphes concrets sur lesquels nous travaillions (pour rappel, jusqu'à 5 couches, jusqu'à 20 000 nœuds par couche, plusieurs centaines de milliers d'arêtes), une itération de l'algorithme `itRWR` prenait plusieurs minutes et cela semblait relever d'une anomalie. Nous visions des bonnes performances pour une dizaine d'itérations, ce qui pouvait correspondre à une demi-heure, selon la machine, pour une marche aléatoire. Ce temps n'était pas tolérable, que ce soit pour les utilisateurs finaux ou pour nous (pour pouvoir effectuer des tests après avoir modifié le code source).

Les MOA souhaitaient également connaître les détails de cette optimisation. C'est pourquoi nous avons veillé à bien développer cette section.

9.1 MÉTHODOLOGIE

Nous avons optimisé le code sur une machine avec un processeur Intel i5-5350U (4 threads) @ 1.740GHz, 8 Go de RAM et un SSD NVMe. Les modifications ont toutes été faites dans la branche `opt`.

Notre méthodologie pour optimiser les performances a été de passer l'intégralité du code relatif à la marche aléatoire (`profiler.views.py.test_itRWR()`) dans [cProfile](#), le profiler natif à python, et de visualiser les résultats avec [snakeviz](#) pour repérer facilement quelles modules et fonctions sont couteuse en temps et pour avoir une bonne vue d'ensemble de la pile d'appels.

Ensuite, pour chaque fonction ou méthode que nous repérions comme couteuse et non native à python, nous avons utilisé nos connaissances en complexité ainsi que [line profiler](#) pour repérer quelles lignes du code étaient les plus couteuses. Avec ces informations, on pouvait soit simplement déplacer un calcul répété inutilement vers l'extérieur d'une boucle, soit trouver une alternative moins couteuse. Par ailleurs, nous nous sommes également aidés de *Deepseek*⁵ pour faciliter l'optimisation des bouts de code repérés.

Notre méthodologie pour écrire des prompts à Deepseek a été d'envoyer le code source de la fonction fautive en précisant quelle était la partie couteuse et, en anglais, s'il fallait, selon le code en question, l'optimiser, la vectoriser ou la paralléliser. Il a parfois fallu rajouter la source des modules "mal compris" lorsque Deepseek ne pouvait pas produire de résultat interprétable.

Pour chaque modification, il a fallu s'assurer que l'on obtenait le même résultat, donc on a soit ajouté des tests de non-régression pour les fonctions modifiées (mais par la suite commentées pour ne pas perdre l'optimisation), en plus de tests à un plus haut niveau dans la pile d'appels. Nous avons conservé la plupart du code d'origine en dupliquant ses fonctions et les renommant avec un suffixe "_old" afin de pouvoir les comparer à nos versions et les tester.

Après chaque modification, on recommençait au départ de notre méthodologie (cProfile avec snakeviz) pour pouvoir vérifier plusieurs fois qu'il y'avait bien une amélioration, et pour repérer les nouveaux principaux coupables.

On avait aussi ajouté un timer (`time.perf_counter`) pour calculer le temps d'exécution sans profileur, car le profilage pouvait influencer sur le temps d'exécution.

9.2 DÉROULEMENT

Nous avons d'abord identifié une de nos propres fonctions comme responsable de mauvaises performances. Il s'agissait d'une fonction que nous avons ajoutée à notre dépendance modifiée itRWR, nommée actuellement `update_layer_max_scores` (anciennement `process_ranking_files`). Celle-ci utilisait en effet la méthode `.iterrows()` qui n'est pas recommandé et qui est une erreur courante de débutant avec pandas (KieranPC, 2014). De plus, l'appel `iterrows` était dans une boucle, ce qui augmente encore la complexité. Dans cette *iteration*, nous avons d'abord repéré la méthode `df.loc` comme étant couteuse à l'aide des profiler, et avons donc essayé d'utiliser son équivalent en numpy, mais nous avons gagné trop peu de temps d'exécution.

⁵ <https://www.deepseek.com>

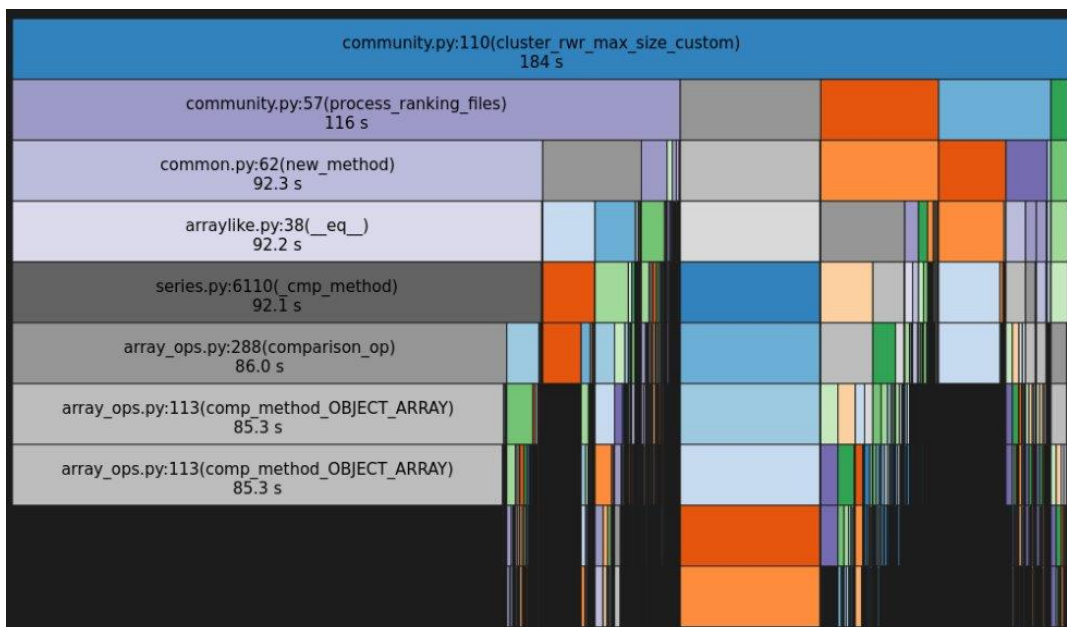


Figure 3: Visualisation Snakeviz du dump cProfile avant optimisation⁶

Nous avons donc appris qu'avec pandas, il faut toujours éviter d'utiliser `.iterrows()`, et veiller à vectoriser les opérations si c'est possible. Comme nous n'étions pas très habitué à pandas, nous avons demandé à Deepseek de produire une version vectorisée de ce code à plusieurs reprises. Nous sommes parvenus à obtenir un résultat satisfaisant en demandant "how to vectorize this instead of iterating, *iterrow is the bottleneck*" suivi du code entre guillemets de la fonction `process_ranking_files` telle qu'elle était au commit `fd05f7507392209c6833eab643350a7e3e108247`.

Dans notre propre code, nous avons aussi supprimé des réécritures de fichiers inutiles (car effacés par le suivant dans une boucle), et enlevé des relectures redondantes de fichiers en les gardant en mémoire ; mais ces deux optimisations avaient un effet négligeable sur le temps d'exécution.

⁶ En abscisse le temps, en ordonnée la pile d'appel du haut vers le bas, chaque fonction étant représenté par un rectangle appelant les rectangle en-dessous de lui. Voir (Davis, 2024) pour l'interprétation des diagrammes

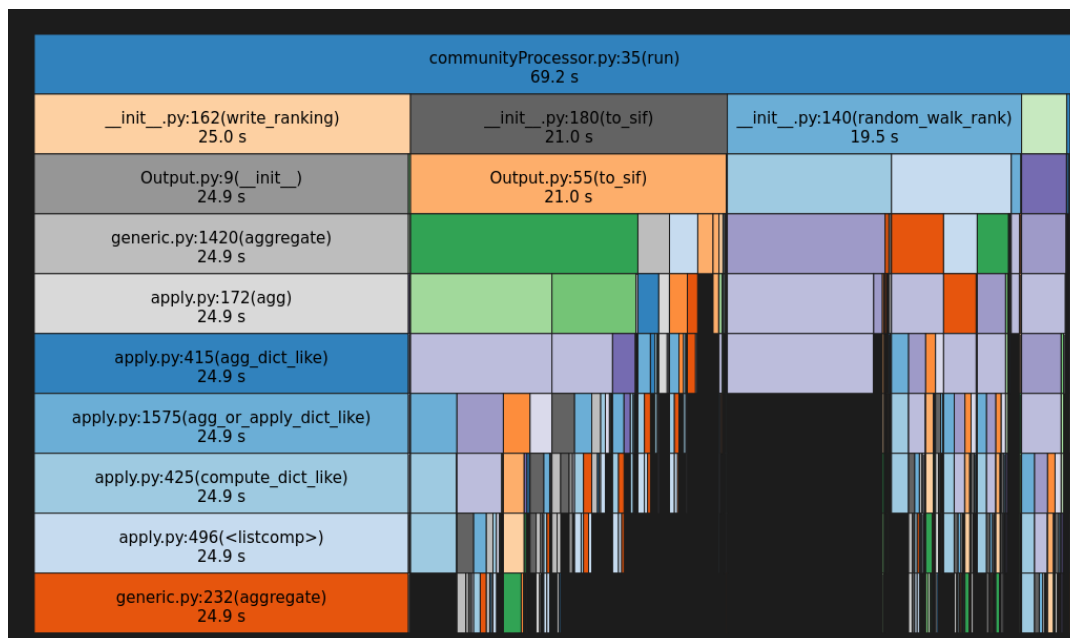


Figure 4: Visualisation snakeviz du dump cProfile au commit `c2176da54e3015cd29f4dfa15f695188c39d2ea7`

Suite à la précédente optimisation, la majorité du temps d'exécution est occupée par la dépendance `multixrank` (cf. figure ci-dessus, toutes les méthodes de la deuxième ligne sont dans `multixrank`). Nous avons d'abord pensé à simplement modifier les fonctions utilisées dans notre propre code mais c'était uniquement possible pour `write_ranking`. Nous avons donc fait une sorte de fork à l'intérieur de notre dépôt comme cela avait été fait pour `itRWR`.

Nous nous sommes d'abord penchés sur `write_ranking` car celui-ci prenait légèrement plus de temps. On a trouvé à l'aide `line_profiler` que l'objet `Output` faisait un calcul de moyenne géométrique peu efficace sur une ligne en utilisant la bibliothèque `scipy.stats`. Toujours à l'aide de l'inspiration de Deepseek, on a trouvé un moyen de calculer cette moyenne sans passer par cette bibliothèque, en plusieurs étapes (voir Figure 5) : On calcule d'abord le logarithme des scores avec `numpy`, puis une moyenne native à `pandas` que l'on exponente ensuite toujours avec l'aide de `numpy`. On obtient alors un résultat identique en beaucoup moins de temps.

```

if (self.aggregation == "gmean"):
    # Filter the DataFrame first
    self._df = self._df.loc[self._df.score > 0].copy()
    # Compute the logarithm of the 'score' column
    self._df['log_score'] = np.log(self._df['score'])
    # Group by and compute the mean of the log scores
    self._df = self._df.groupby(['multiplex', 'node']).agg({'log_score': 'mean'}).reset_index()
    # Exponentiate to obtain the geometric mean and clean up
    self._df['score'] = np.exp(self._df['log_score'])
    self._df = self._df.drop(columns=['log_score'])
if (self.aggregation == "gmean_old"):
    self._df = (self._df.loc[self._df.score > 0]).groupby(['multiplex', 'node'])
    ||.agg({'score': mstats.gmean}).reset_index()

```

Figure 5: Modification dans Output.py

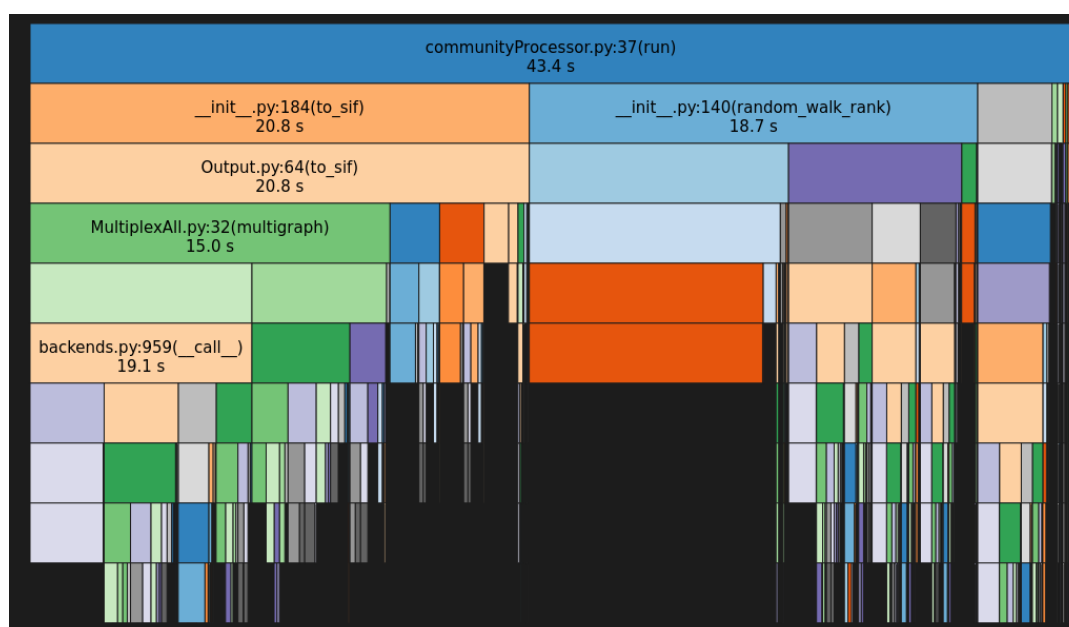


Figure 6: Visualisation du dump cProfile au commit 616a3f1287b5833397484bb52e61187dddbd7374

L'optimisation de `to_sif` a été moins fructueuse même si nous avons réduit son temps par deux, elle a été aussi plus coûteuse en temps de développement comme nous avons persisté à vouloir améliorer cette méthode. Celle-ci repose sur la bibliothèque `networkx` pour convertir les données et les exporter des graphes vers le format SIF. Cette bibliothèque est la principale cause du manque de performance de cette méthode. Nous avons aussi retrouvé un appel à la méthode optimisée précédemment, mais elle a peu d'impact ici.

Nous avons tout de même modifié chaque conversion en `networkx.Graph` dans les propriétés `Multiplex.networkx` et `Bipartite.networkx`, pour les rendre légèrement plus performantes et parallélisées (`MultiplexAll.py`) afin d'avoir au plus une tâche de conversion pour chaque couche ou pour chaque cœur du processeur. On a aussi remplacé l'utilisation de `networkx.add_edges_from` par une fonction légèrement plus performante, et l'ajout de nœuds ou d'arêtes complètement en parallèle ne semblait pas

possible. Plutôt que de chercher à optimiser `.to_sif` (cette étape était inutile, mais nous ne comprenions pas certaines opérations dans `.to_sif` et n'étions pas assez à l'aise avec pandas pour être sûrs de ne pas passer trop de temps à les réécrire), nous aurions dû

- Soit remplacer toute la dépendance à `networkx` par une bibliothèque plus performante.
- Soit passer le `dataFrame` de la dépendance à `itRWR`.

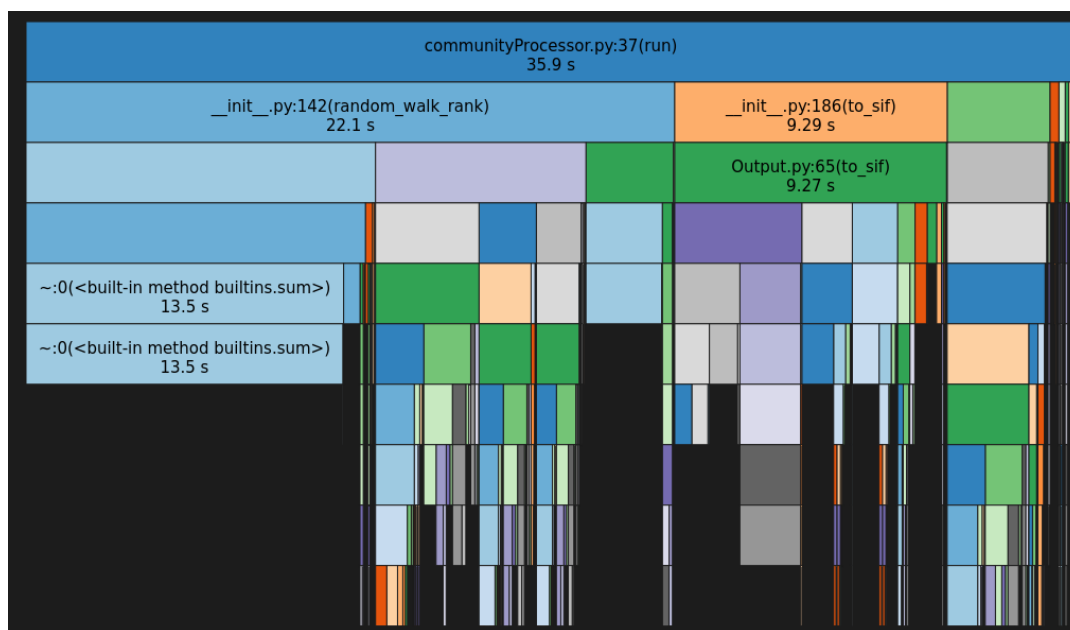


Figure 7: Visualisation du dump cProfile au commit `od38298e9c71e567073674bfbf83691e6edfd4df`

Enfin, nous nous sommes attaqués à l'algorithme de marche aléatoire qui paraissait plus compliqué à tester comme nous supposions qu'il devait produire un résultat aléatoire. Cependant nous avons trouvé qu'initialiser deux objet `Multixrank` avec les mêmes paramètres et appeler leurs méthodes de marche aléatoire devait produire le même résultat.

Nous avons découvert avec l'aide de cProfile que la méthode `get_normalisation_alpha_alpha` de `TransitionMatrix` prenait le plus de temps, et nous avons remarqué que le manque de performance était lié au recalcul en boucle d'une même somme, en plus des appels en boucle de la méthode elle-même. Nous avons donc simplement déplacé le calcul de la somme en dehors de la boucle, car sa valeur ne pouvait pas changer.

L'autre responsable était `BipartiteAll.bipartite_matrix`. Son code était assez illisible, et la ligne couteuse était la suivante :

```
B = networkx.to_scipy_sparse_array([...])
```

Nous avons fait appel à Deepseek et nous avons dû lui fournir les codes sources de `Multiplex.py` et `Bipartite.py` pour avoir en sortie du code cohérent. La partie la moins lisible reste peu lisible, mais nous lui avons aussi fait produire une version lisible selon Deepseek de l'ancien code, qui n'est pas plus performante, mais produit le même résultat.

Nous avons aussi tenté d'optimiser `__random_walk_restart` qui est réellement le coeur de l'algorithme de marche aléatoire et est relancé à chaque itération mais sans succès. Il ne semble pas y avoir d'amélioration facilement identifiable, mais on pourrait peut-être trouver le moyen de vectoriser ce code.

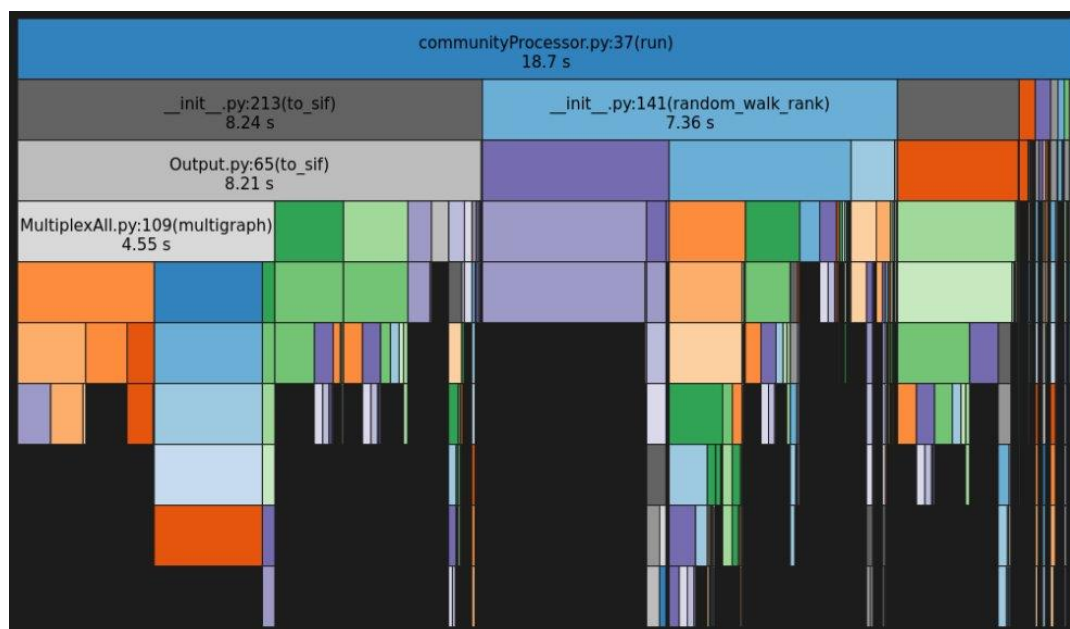


Figure 8: Visualisation du dump cProfile après les dernières optimisations

Il est sûrement encore possible d'améliorer les performances de la marche aléatoire, mais nous avons tout de même divisé le temps d'exécution de l'ensemble par 10, et cela semble plus que suffisant pour répondre au besoin non fonctionnel de performance d'autant plus que les MOAs n'ont pas insisté sur cet aspect et que nous devons également nous concentrer sur d'autres tâches de priorité plus élevée.

9.3 POINTS DÉLICATS

Nous n'avons pas cherché à optimiser les multiples écritures et lecture redondantes sur disque, car nous avons testé le programme uniquement sur SSD et ces opérations nous semblaient peu coûteuses par rapport aux défauts de performances de calcul. Comme nous avons commencé à modifier la dépendance, nous aurions pu court-circuiter tous les passages par des fichiers, et toutes les conversions entre la dépendance (finalement modifiée et intégrée) et notre programme (notamment la conversion d'un `dataFrame`

pandas en graph networkx puis en fichier SIF qui est ensuite réimporté par notre programme et converti en graph networkx). Il nous semblait cependant plus simple et plus rapide de simplement optimiser la dépendance. Nous avons donc fait cela en priorité. De plus, le code d'importation aurait dû être supprimé, et le `.to_sif` comportait des étapes en plus des conversions que nous n'étions pas sûrs de comprendre. Cela reste néanmoins une piste pour continuer à optimiser le programme et l'on pourrait gagner environ un à deux tiers du temps d'exécution (cf. "`.to_sif`" profilage final).

10 ANALYSE DES SPRINTS

10.1 SPRINT 1 (17 AU 28 FÉVRIER 2025)

Lors de ce premier sprint, nous avons établi une première liste des besoins pour notre application, en en surestimant quelque peu le nombre (par exemple, nous avons inclus des enjeux peu pertinents pour une POC comme la sécurité). Cette liste a fortement évolué en cours de sprint, et comme nous ne pratiquons pas encore le daily Scrum et n'avons pas désigné de sprint master, la gestion du projet a été quelque peu erratique. Toutefois, nos avancées sur le projet ont été plus rapides que prévues, et nous avons pu développer un premier backend et frontend fonctionnels, même si le lien entre les deux était encore absent.

Nous avons déjà réalisé un prototype d'interface graphique, et nous avons pu préciser avec les MOA les détails d'exécution de l'algorithme ainsi que les caractéristiques attendues pour le graphe de sortie.

Les deux principales aires de difficulté que nous pensions avoir identifiées concernant l'algorithme, la reconstruction des chemins et l'affichage des couches sous forme de tranches, ont été résolues en très peu de temps.

Nous avons eu des difficultés pour obtenir un environnement de programmation opérationnel (que ce soit chez nous ou au CREMI).

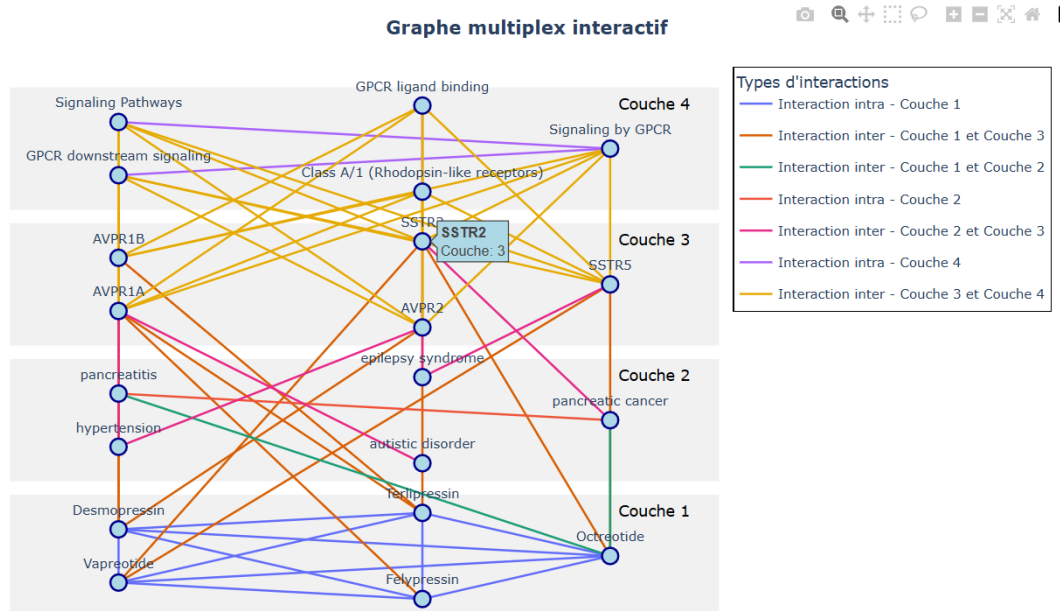


Figure 9: Graphe généré lors du premier sprint

Lors du sprint review le 28/02, les MOA nous ont dit qu'elles étaient *ravies*, tout en nous faisant de nombreuses suggestions d'améliorations, notamment concernant l'affichage du graphe de sortie.

Sur ce premier sprint, nous découvrons encore l'interface de Jira, donc les estimations de story points n'ont été effectuées qu'après plusieurs jours.

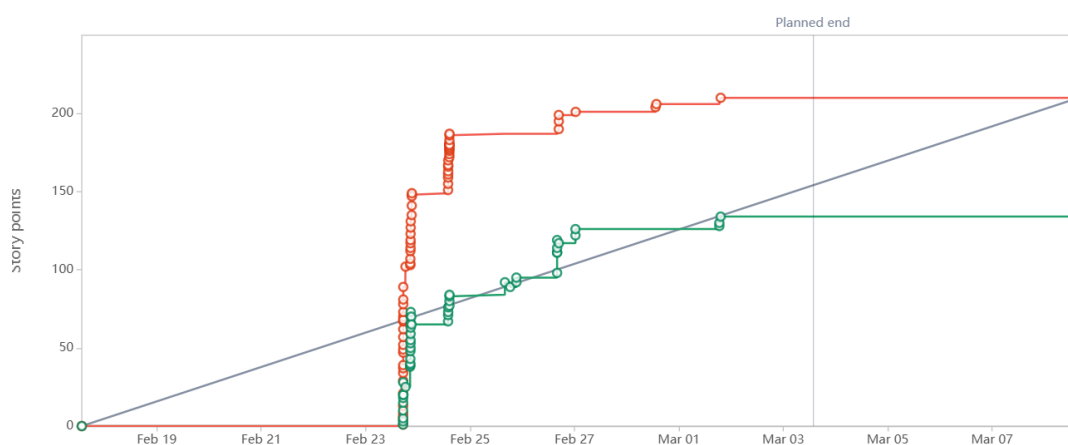


Figure 10: Burnup du premier sprint

10.2 SPRINT 2 (10 AU 16 MARS 2025)

Comme trois semaines s'étaient écoulées depuis le début du premier sprint, et comme notre compréhension du projet avait beaucoup évolué, nous avons décidé pour la planification de ce sprint de remettre le backlog sur le métier intégralement⁷, en posant pour chaque ticket encore ouvert les questions suivantes :

- Ne l'a-t-on pas résolu sans s'en rendre compte ?
- Est-il toujours pertinent ?
- Ne fait-il pas doublon avec d'autres items ?
- Est-il correctement décrit et compréhensible ?

Après avoir effectué ce tri, les catégories de tâches suivantes se sont dessinées :

- Terminer l'intégration back/front
- Implémenter les suggestions des MOA
 - Ajouter la possibilité de personnaliser les paramètres
 - Améliorer l'UI
 - Améliorer l'affichage des graphes
- Implémenter une base de données pour le stockage des résultats
- Écrire des tests
- Écrire la documentation
- Optimiser l'algorithme

Le backlog était prévu pour deux semaines, mais nous avons ensuite décidé d'effectuer à la place deux sprints d'une semaine chacun.

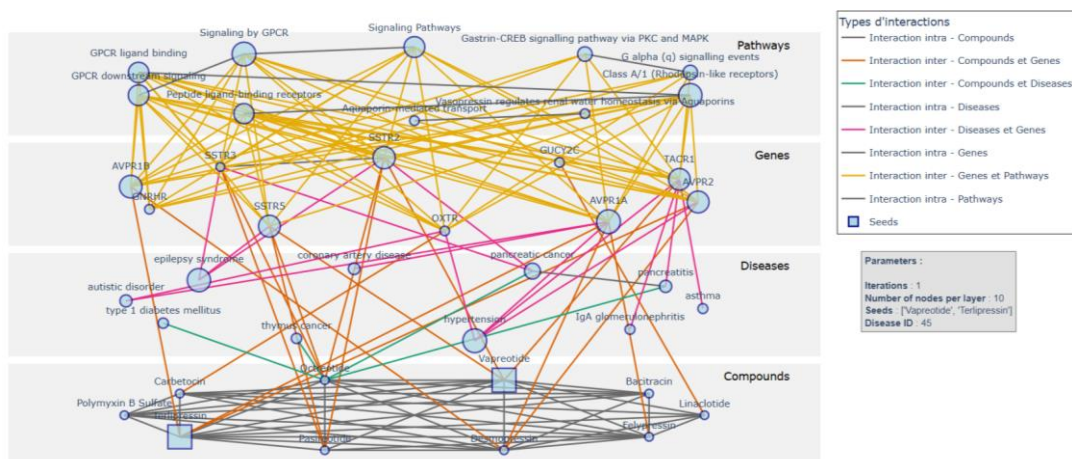


Figure 11: Graphe généré lors du second sprint

⁷ Une archive de nos backlogs de sprints (ainsi que de nos notes de réunions hebdomadaires avec le MOA) est disponible dans le dossier **documents** du dépôt.

La fixation d'une date pour une première démo publique (le 19 mars) nous imposait de nous concentrer sur la disponibilité d'un produit démontrable (donc ayant une UI entièrement fonctionnelle et un backend et un frontend complètement intégrés).

Malgré tout cela, en fin de sprint seules deux catégories de tâches (tests et documentation) restaient inachevées.

Lors de la réunion du 13 mars, les MOA nous ont indiqué que nous avions *validé toutes leurs demandes*.

Date - 10 mars 2025 - 15 mars 2025

Objectif du sprint - Compléter l'intégrité du projet (intégration, tests, logs, doc), implémenter les suggestions du MoA, optimiser l'algorithme



Figure 12: Burnup du second sprint

10.3 SPRINT 3 (17 AU 21 MARS 2025)

Nous devons donc nous concentrer pour ce dernier sprint sur la complétude des tests et de la documentation, ainsi que sur le déploiement de l'application (qui n'est pas *a priori* amenée à être hébergée sur un serveur).

Malgré quelques hics de compilation (essentiellement dus à la connexion Wi-Fi), notre démo du 19 mars a été un succès. Pour la première fois, nous avons pu tester la pertinence de notre outil dans un contexte bio-informatique, et sa capacité à faire apparaître des liens qui étaient moins apparents dans les données initiales.

Bien que nous ne prévoyions plus à ce stade d'ajouter des fonctionnalités, nous avons accepté d'implémenter quelques modifications suite aux retours durant la démo.

Le travail effectué pendant ce sprint a été essentiellement attribué à la résolution des derniers bugs et à la refactorisation du code. Nous n'avons pas eu le temps d'écrire des tests pour certaines fonctions du frontend.

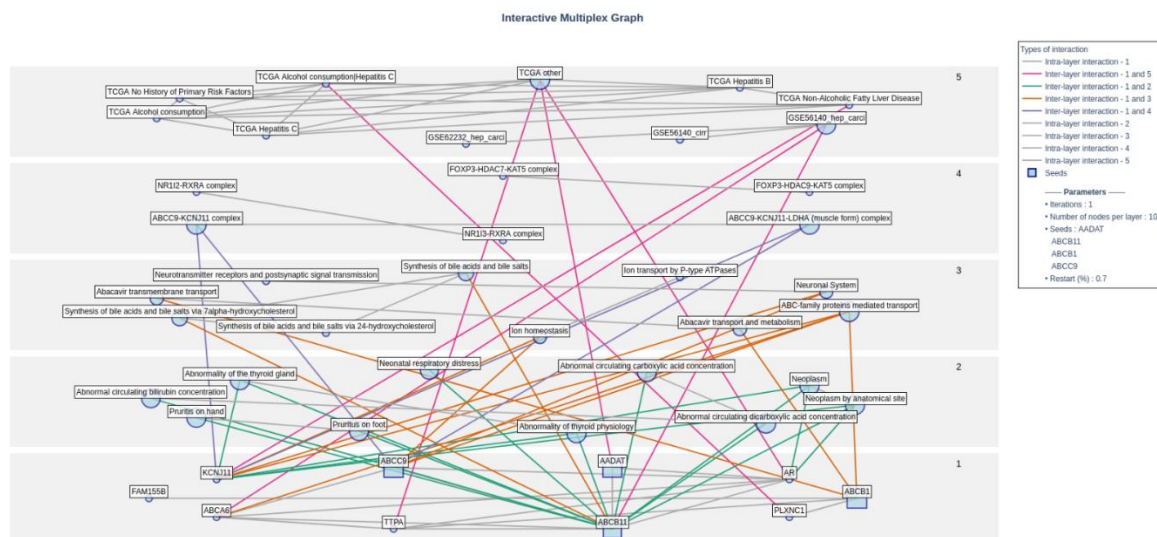


Figure 13: Graphe généré en fin de sprint 3

Date - 16 mars 2025 - 21 mars 2025

Objectif du sprint - Release entièrement testée et documentée

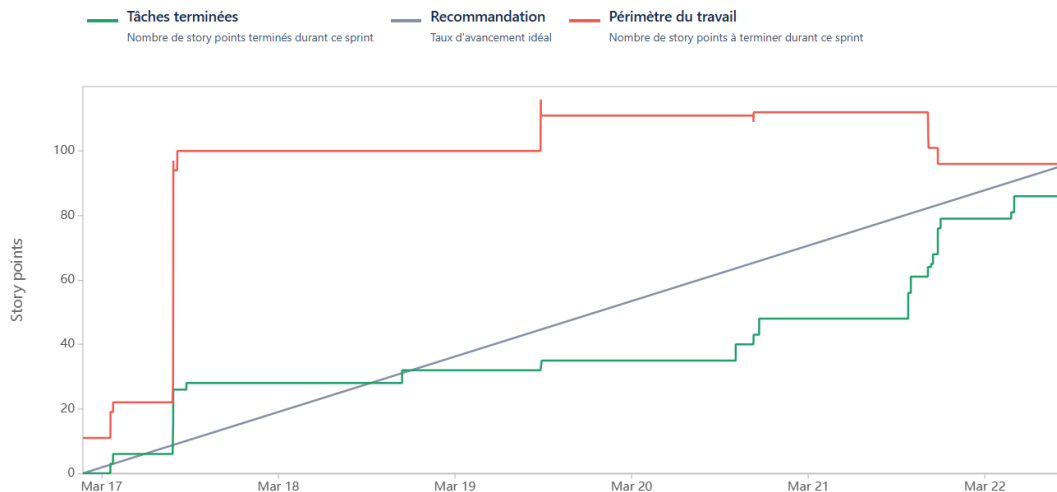


Figure 14: Burnup du troisième sprint

10.4 SPRINT 4 (24 AU 26 MARS 2025)

Il s'agissait essentiellement de régler les derniers problèmes que nous avons repérés, ainsi que de corriger le rapport. Nous n'avons pas effectué de planification formelle pour ce sprint, mais nous sommes contents d'ouvrir des tickets au fur et à mesure que nous repérons des points d'amélioration. De plus, plusieurs tickets avaient en réalité été achevés entre la fermeture du sprint précédent et l'ouverture de celui-ci.

Le burnup pour ce sprint est encore incomplet, car il contient un ticket pour l'écriture du rapport.

Date - 23 mars 2025 - 26 mars 2025

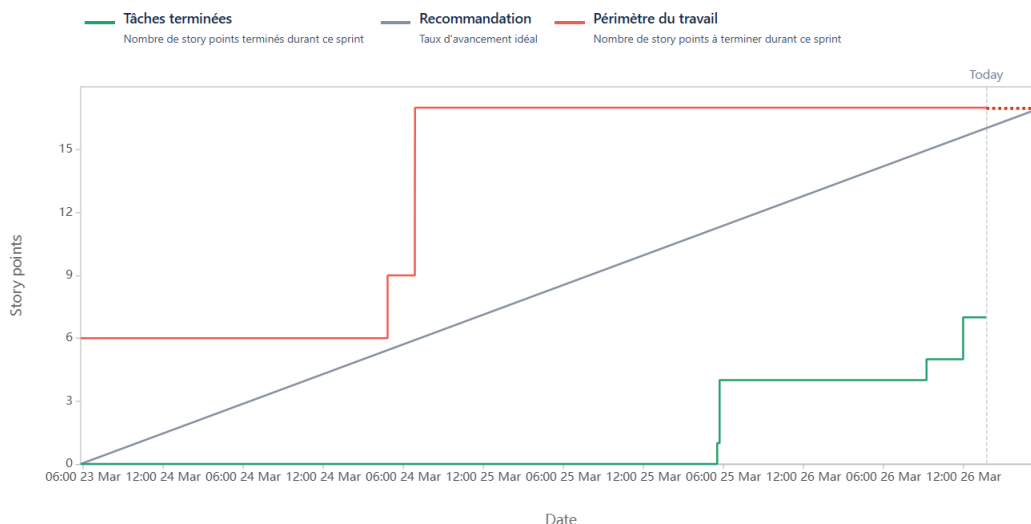


Figure 15: Burnup du sprint 4

11 RETOURS SUR SCRUM ET SON APPLICATION

L'application de Scrum nous a permis de pouvoir continuellement remettre le métier sur l'ouvrage, en intégrant au fur et à mesure les retours clients et en faisant évoluer notre planification de façon à la garder pertinente. Cela nous a également permis de ne pas se fixer sur l'accomplissement d'une liste de tâches conçue à un stade embryonnaire du projet, lorsque notre idée des besoins ou des difficultés de chaque tâche était encore très peu précise.

Toutefois, les fenêtres temporelles des sprints ont pu être parfois arbitraires. Les dates les plus importantes n'étaient pas les fins de Sprint, mais les réunions avec les MOA ainsi que les préparations d'audits. Il aurait peut-être été plus pertinent de définir les sprints comme étant le temps entre deux réunions par exemple.

Ces réunions avaient lieu à un rythme hebdomadaire, et nous ont permis de préciser les attendus de l'application. Notamment, la description initialement proposée sur le site du PFE était assez différente de ce qui était réellement attendu (par exemple il n'avait jamais été question d'afficher un graphe entier, seulement un sous-graphe après marche aléatoire).

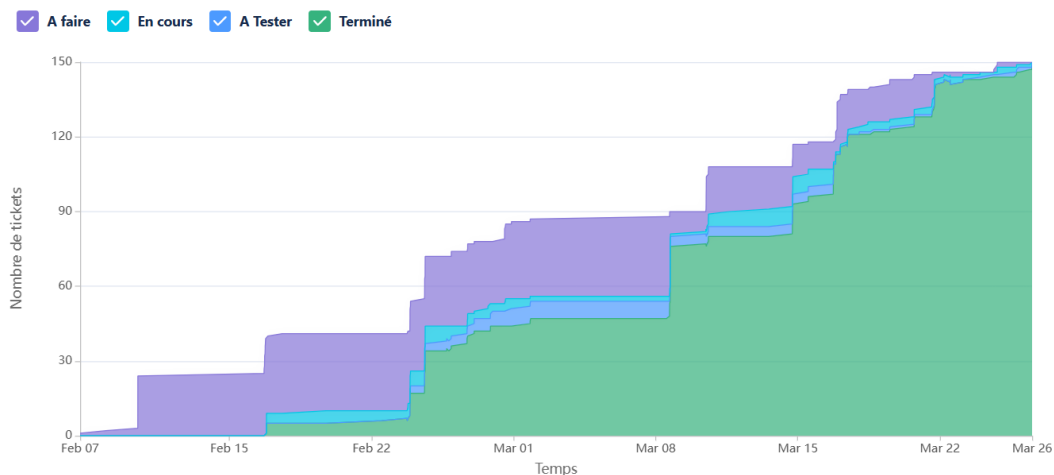


Figure 16: Diagramme de flux cumul  du projet

Pendant tout le projet, nous avons d fini le p rim tre de travail (story points) au d but de chaque sprint, plut t qu'  la fin du sprint pr c dent. Cela nous permettait d'avoir l g rement entam  le travail et donc d'effectuer une estimation inform e. Nous modifions  galement cette estimation quand  a s'av rait n cessaire (t che plus simple ou plus compliqu e que pr vu).

Toutefois, ce mode de travail a l'inconv nient de rendre difficilement lisibles certains diagrammes propos s par Jira (en particulier le burndown et le diagramme de v locit ), donc nous avons choisi de n'inclure dans ce rapport que les diagrammes suffisamment lisibles et pertinents.

12 BIBLIOGRAPHIE

- Baptista, A., Beust, C., & Baudot, A. (s.d.). *itRWR: MultiXrank-based Community Identification with Iterative Random Walk with Restart*. Consult  le 03 24, 2025, sur Github: <https://github.com/anthbapt/itRWR>
- Baptista, A., Gonz lez, A., & Baudot, A. (2022, Juillet 1). Universal multilayer network exploration by random walk with restart. *Nature*, p. 170.
- Davis, M. (2024). *Snakeviz # Interpreting Results*. Consult  le 03 25, 2025, sur Jiffyclub: <https://jiffyclub.github.io/snakeviz/#interpreting-results>
- KieranPC. (2014, 07 21). *Does pandas iterrows have performance issues?* Consult  le 03 19, 2025, sur Stackoverflow: <https://stackoverflow.com/questions/24870953/does-pandas-iterrows-have-performance-issues>

13 ANNEXE

13.1 TABLEAU DES TESTS RÉALISÉS CÔTÉ BACKEND

Nom du test	Domaine d'application	Domaine d'utilisation	Description
test_create_arbo_from_zip.py	Structure de fichiers ZIP (arborescence)	Extraction automatique d'un ZIP fourni par l'utilisateur	Teste l'extraction correcte d'un ZIP structuré contenant des fichiers et dossiers.
test_file_utils.py	Fichiers de configuration et lecture de graines	Lecture ou écriture de fichiers utilisés en préparation par l'utilisateur	Vérifie la lecture/l'écriture des fichiers de config et de graines pour l'algo.
test_generate_config.py	Génération de configuration à partir de paramètres	Création automatique d'un fichier config pour lancer l'algo selon les paramètres utilisateur	Génère un fichier YAML de configuration à partir d'un ZIP, des graines et paramètres.
test_get_zip_file.py	Détection de fichiers ZIP dans un dossier	Détection du fichier ZIP sélectionné par l'utilisateur pour analyse	Vérifie qu'on détecte correctement un ZIP ou qu'on gère l'erreur s'il est absent.
test_list_layers.py	Extraction de couches d'un graphe multiplex	Lecture des couches du graphe sélectionné par l'utilisateur	Vérifie la lecture correcte des couches depuis un ZIP (fichiers multiplex).
test_list_unique_nodes.py	Extraction de nœuds d'un graphe	Identification des nœuds accessibles à l'utilisateur dans une couche	Teste l'identification de nœuds uniques dans les fichiers du graphe multiplex.
test_ranking_utils.py	Résultats de classement (marche aléatoire)	Mise à jour dynamique des graines pendant l'exécution selon itérations	Vérifie le traitement et l'actualisation des résultats de classement sur les nœuds.

<code>test_sif_utils.py</code>	Fichiers SIF (export/visualisation)	Préparation de fichiers exportables ou visualisables par l'utilisateur	Teste la correction de l'ordre des lignes SIF et leur filtrage selon les couches.
<code>test_views.py</code>	Interface API web	Interaction de l'utilisateur avec l'API (via l'interface web)	Teste que l'API renvoie des réponses correctes (ex. liste des multiplexes, couches, etc.).
<code>test_write_config_file.py</code>	Mise à jour de fichier de configuration	Mise à jour automatique de la config suite à l'entrée des paramètres utilisateur	Vérifie que les chemins dans le fichier de config sont bien modifiés selon l'ID/seed.
<code>test_get_layer_mapping.py</code>	Lecture de correspondances entre ID de couches et noms	Identification des couches disponibles à partir d'une archive utilisateur	Teste la lecture du fichier <code>layer_name.tsv</code> pour en extraire un mapping couche-ID.
<code>test_get_layer_names.py</code>	Lecture de noms de couches depuis un fichier de configuration	Chargement et affichage des couches disponibles à l'utilisateur	Teste l'extraction des noms de couches dans un dossier de configuration.

13.2 TABLEAU DES TESTS RÉALISÉS CÔTÉ FRONTEND

Nom du test	Domaine d'application	Domaine d'utilisation	Description
test_selenium_upload_zip.py	Test fonctionnel de la page web d'upload des données du graphes.	Validation du processus d'upload un fichier zip via le formulaire.	Vérifier l'upload d'un fichier ZIP local, S'assurer que le fichier sélectionné est envoyé via l'élément de formulaire prévu. Valider que, suite à l'action d'upload, l'application redirige vers la bonne page.
test_selenium_select_seed.py	Test fonctionnel de la page web de sélection des graines.	Validation du processus de sélection de graine et de génération de graphique.	Vérifier que tous champs de formulaire soit modifiable, S'assurer que l'on puisse choisir et valider des graines, Valider qu'un nouvel onglet s'ouvre et contient un script Plotly
selenium_layer_selection.py	Test fonctionnel de la page web de sélection de couche du graphe multiplex.	Validation de la redirection et de la configuration de la page de choix après upload d'un fichier ZIP et sélection d'une couche.	Vérifier que l'on puisse choisir une couche de la liste déroulante. Valider que, suite à l'action de confirmation, l'application redirige vers la bonne page.
test_selenium_log.py	Test fonctionnel de la page web de gestion de logs.	Vérification end-to-end des actions de téléchargement, visualisation et suppression d'entrées d'historique.	Générer un graphe pour s'assurer d'avoir au moins un élément dans l'historique, Vérifier que chaque bouton réalise l'action de son icône, Valider que le bouton de téléchargement télécharge bien un fichier, que supprimer un graphe le retire de l'affichage, que visualiser un graphe ouvre un onglet avec scripts Plotly

13.3 DIAGRAMME DE FLUX DE L'APPLICATION

